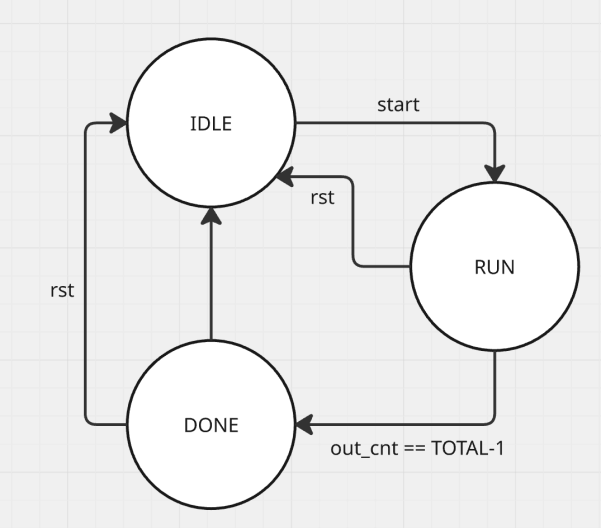
PERANCANGAN RTL CONVOLUTION ENGINE DENGAN RELU DAN MAXPOOL

Danny Rahmat Pratama (13224019)
Tanggal Pengumpulan: 11/10/2026
EL4013 - Perancangan SIstem VLSI
Sekolah Teknik Elektro dan Informatika ITB

1. Desain Sistem

1.1 Diagram FSM

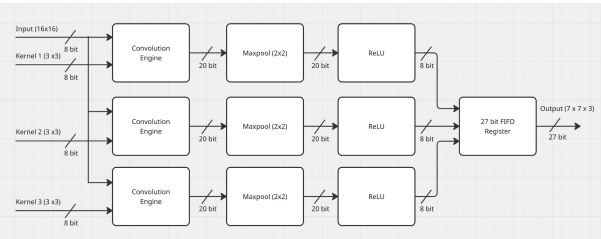


Gambar 1-1 Diagram FSM

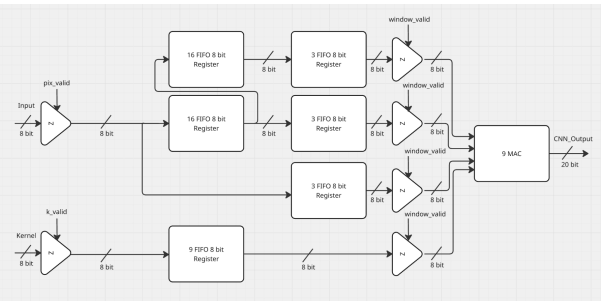
FSM berbentuk sederhana dimana terdiri dari lima *state*, yaitu:

1. IDLE, menunggu input start untuk memulai operasi konvolusi dan melakukan *loading* kernel.
2. RUN, melakukan perhitungan konvolusi, maxpooling, dan ReLU, hingga mendapatkan seluruh hasil.
3. DONE, output telah dikeluarkan semua dan kembali ke state IDLE

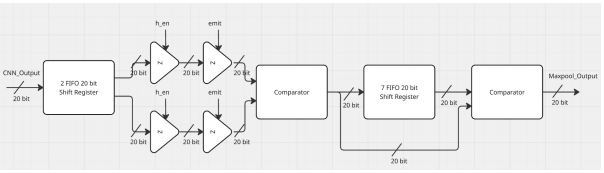
1.2 Skematik Datapath



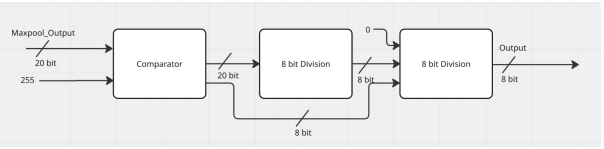
Gambar 1-2 Top Level Datapath



Gambar 1-3 Datapath untuk Convolution Engine



Gambar 1-4 Datapath untuk Maxpool



Gambar 1-5 Datapath untuk ReLU

Pada datapath dibagi menjadi empat, yaitu:

1. Top Level Datapath, pada bagian ini terlihat input dan output serta *building blocks* dari sistem yang digunakan.
2. Convolution Engine Datapath, bagian ini memperlihatkan datapath untuk convolution engine, mulai dari metode *sliding window* dan 9 *multiple and accumulator* yang dilakukan secara parallel.
3. Maxpool Datapath, pada bagian ini memperlihatkan datapath untuk maxpooling yang menggunakan dua comparator untuk membandingkan nilai vertikal dan horizontal.
4. ReLU Datapath, pada bagian ini digunakan komparator untuk membandingkan nilai output sehingga mendapatkan output yang terkuantisasi dan sesuai dengan nilai ReLU.

Desain yang digunakan menggunakan pendekatan optimisasi area dan juga *maximum throughput*. Dalam hal ini optimisasi area merujuk pada penggunakan *resource* untuk register yang menggunakan *line buffer* di input *convolution* dan di input *maxpool*, sehingga tidak menyimpan seluruh data melainkan hanya menggunakan metode *window sliding* untuk mendapatkan data yang diinginkan. Untuk di maxpool sendiri, menggunakan *partial result* sehingga membagi output menjadi dua untuk mengurangi penggunaan *resource* register.

Kemudian untuk *maximum throughput* merujuk pada penggunaan *pipelined* MAC yang melakukan 9 input secara paralel untuk menghasilkan nilai konvolusi.

1.3 Deksripsi Modul Verilog

Tabel 1-1 Modul Verilog

Modul Verilog	Deskripsi
controller	Bagian yang mengontrol FSM yang terdiri dari tiga state, yaitu IDLE, RUN, dan DONE.
conv_pool_relu_top	Bagian ini adalah top level yang menggabungkan *building block* yang telah dibuat.
conv_ctrl	Bagian ini mengontrol *sliding window* 3x3 untuk mendapatkan input yang valid menuju 9 *multiple and accumulator*.
kernel_loader	Bagian ini memiliki tujuan untuk mengambil nilai dari kernel dan menyimpannya dalam register 3x3.
mac9	Bagian ini adalah 9 buah *multiple and accumulator* yang melakukan konvolusi antara kernel dan input untuk secara parallel.
pool_ctrl	Bagian ini mengontrol maxpool untuk mengontrol bagian yang masuk ke dalam 2 register.
pool_relu_ch	Bagian ini adalah bagian yang menggabungkan maxpool dan ReLU.
relu_sat	Bagian komparator yang digunakan untuk ReLU
shift_reg	Shift register yang digunakan untuk menyimpan data pada *building blocks*
signed_mult	Bagian ini adalah bagian *multiple* pada MAC yang bertujuan untuk mengalikan signed dan unsigned.
window_frontend	Digunakan untuk menyimpan input secara paralel yang dapat digunakan untuk beberapa building blocks.

Pada implementasi kode di Verilog, dibuat beberapa *building blocks* terlebih dahulu untuk menghasilkan *building blocks* diatasnya. Sehingga dilakukan pendekatan bottom-up. Pada pendekatan ini dibuat beberapa *building blocks* tingkat rendah yaitu shift_reg dan signed_mult. Kemudian dari kedua tersebut, dibuat *building block* tingkat menengah, yaitu pool_ctrl, window_frontend, kernel_loader, relu_sat, dan mac9. Setelah itu baru dibuat *building block* tingkat tinggi, yaitu pool_reul_ch, controller, dan conv_ctrl. Terakhir dibuat top level, yaitu conv_pool_relu_top.

2. Hasil Simulasi dan Analisis Timing

2.1 Golden Model dan Hasilnya

Tabel 2-1 Kode golden_custom.py

```
"""Golden model + testbench generator for the specific input given in the task.
Run:  python3 golden_custom.py      -> prints all intermediate stages, writes
tb_custom.v
"""
import numpy as np

img = np.array([
 [170,249, 18,218,  4,172,206, 45,191, 76,195,  6,103,
89,209,163],
 [234,241,143, 93,229,230,158,119,115,156,111, 20, 95, 31,217,
94],
 [ 26, 54,185,164,199,220,219, 90, 40,107,192,212, 92,103,
95,202],
 [ 92, 91, 85,180,168,129,183,231, 23, 32,161,184,206,  4,103,
45],
 [ 79,179,195,219,162,231, 28,167, 13, 29, 53, 25,246,145, 93,
76],
 [ 49,148,105, 42,178,213,138,247,113,150, 91,186,212,
76,123,235],
 [ 24,221, 81, 72,212,109,200,162,  0, 41,171,154,251,
12,179,192],
 [ 97, 80, 54, 30,214, 28,225,131, 53,201,  0,165,
36,223,109,211],
 [ 20, 73,245,167, 66, 17,201,114, 27, 94,203,117,182,105,188,
86],
 [ 13,219, 64, 40, 42,103, 36, 71,151,151,104, 37,149,246,240,
57],
 [218,119,146,218,193,238,155,134,118, 34,115, 23,184, 64,
80,110],
 [ 26, 51, 82, 69,132, 52,198,169,115,127,142,212,139,131, 73,
3],
 [ 65,222, 63,119,129,151,156,114,187,150,158,253, 45,125,124,
93],
 [ 88,199, 75, 42,189,147,  5, 50,162,214, 64,122, 53, 33,
73,152],
 [233,217,153, 83,216,174,112,111,  6,184,204,169, 65, 62,204,
76],
 [156,215,248,246,127, 75,156, 53, 91, 50,108,239,185,
92,231,223]], dtype=np.int64)

kers = np.array([
 [[ -28,-111, 125],[ 116,   4,  -3],[ -90,  76,   7]],
 [[-100, -38,   7],[ -53, -45, -47],[  27, -64, 122]],
 [[ 105, -76, 115],[   2, -40, -40],[  59,  60,  16]]],
dtype=np.int64)

SHIFT = 8      # output quantization: ReLU, then >> SHIFT, then saturate to
255

assert img.shape == (16,16) and kers.shape == (3,3,3)

def conv(a, k):
    return np.array([[(a[r:r+3, c:c+3] * k).sum() for c in range(14)] for r
in range(14)])

def pool(m):
    return m.reshape(7,2,7,2).max(axis=(1,3))

conv_o = [conv(img, k) for k in kers]
pool_o = [pool(m) for m in conv_o]
relu_o = [np.clip(m >> SHIFT, 0, 255) for m in pool_o]   # negatives -> 0

np.set_printoptions(linewidth=200)
for c in range(3):
    print(f"=== Channel {c+1} ===")
    print("conv (14x14), min/max =", conv_o[c].min(), conv_o[c].max());
print(conv_o[c])
    print("maxpool (7x7):");  print(pool_o[c])
    print("relu+sat (7x7):"); print(relu_o[c])

# ---- emit a readable, self-contained testbench ----
def row_args(vals, w=3):
    return ", ".join(f"{int(v):{w}d}" for v in vals)
```

```
L = []
A = L.append
A("`timescale 1ns/1ps")
A("//
============================================================
====")
A("//  tb_custom.v  --  directed test: Conv(3 kernels) + MaxPool 2x2 +
ReLU")
A("//  AUTO-GENERATED by golden_custom.py (edit the numbers there, not
here)")
A("//")
A("//  Test flow:  reset -> load 3 kernels -> start -> stream 16x16 image")
A("//                -> collect 7x7x3 outputs -> compare with golden model")
A("//
============================================================
====")
A("module tb_custom;")
A("")
A("    // ---------------- constants ----------------")
A("    localparam N      = 16;           // image size")
A("    localparam POOLN = 7;             // output size per channel")
A("    localparam NOUT  = POOLN*POOLN; // 49 outputs per channel")
A(f"    localparam SHIFT = {SHIFT};           // output quantization: ReLU,
>> SHIFT, saturate")
A("")
A("    // ---------------- DUT signals ----------------")
A("    reg          clk = 0, rst = 1, start = 0;")
A("    reg          k_valid = 0, pix_valid = 0;")
A("    reg  [7:0]   k_data = 0, pix_data = 0;")
A("    wire         out_valid, done;")
A("    wire [23:0] out_data;             // {ch3, ch2, ch1}")
A("")
A("    conv_pool_relu_top #(.N(N), .SHIFT(SHIFT)) dut (")
A("        .clk(clk), .rst(rst), .start(start),")
A("        .k_valid(k_valid), .k_data(k_data),")
A("        .pix_valid(pix_valid), .pix_data(pix_data),")
A("        .out_valid(out_valid), .out_data(out_data), .done(done));")
A("")
A("    always #5 clk = ~clk;             // 100 MHz")
A("")
A("    // ---------------- test data storage ----------------")
A("    reg [7:0] img       [0:N*N-1];        // image, row-major")
A("    reg [7:0] kern      [0:26];           // 3 kernels x 9 weights
(two's complement)")
A("    reg [7:0] exp_val   [0:3*NOUT-1];     // expected, index = ch*49 +
r*7 + c")
A("    reg [7:0] got_val   [0:3*NOUT-1];     // captured from the DUT")
A("")
A("    integer i, ch, r, c;")
A("    integer n_out  = 0;")
A("    integer errors = 0;")
A("")
A("    // ---------------- helper tasks to fill the data
----------------")
A("    task set_row(input integer row,")
A("                 input integer p0, p1, p2, p3, p4, p5, p6, p7,")
A("                 input integer p8, p9, p10, p11, p12, p13, p14,
p15);")
A("        begin")
A("            img[row*N+0]=p0;   img[row*N+1]=p1;   img[row*N+2]=p2;
img[row*N+3]=p3;")
A("            img[row*N+4]=p4;   img[row*N+5]=p5;   img[row*N+6]=p6;
img[row*N+7]=p7;")
A("            img[row*N+8]=p8;   img[row*N+9]=p9;   img[row*N+10]=p10;
img[row*N+11]=p11;")
A("            img[row*N+12]=p12; img[row*N+13]=p13; img[row*N+14]=p14;
img[row*N+15]=p15;")
A("        end")
A("    endtask")
A("")
A("    task set_kernel(input integer k,")
A("                    input integer a, b, c3,")
A("                    input integer d, e, f,")
A("                    input integer g, h, i3);")
A("        begin")
A("            kern[k*9+0]=a; kern[k*9+1]=b; kern[k*9+2]=c3;")
A("            kern[k*9+3]=d; kern[k*9+4]=e; kern[k*9+5]=f;")
A("            kern[k*9+6]=g; kern[k*9+7]=h; kern[k*9+8]=i3;")
A("        end")
A("    endtask")
A("")
A("    task set_exp(input integer chn, input integer row,")
A("                 input integer v0, v1, v2, v3, v4, v5, v6);")
A("        begin")
A("            exp_val[chn*NOUT+row*POOLN+0]=v0;
A("            exp_val[chn*NOUT+row*POOLN+1]=v1;")
A("            exp_val[chn*NOUT+row*POOLN+2]=v2;
exp_val[chn*NOUT+row*POOLN+3]=v3;")
A("            exp_val[chn*NOUT+row*POOLN+4]=v4;
exp_val[chn*NOUT+row*POOLN+5]=v5;")
A("            exp_val[chn*NOUT+row*POOLN+6]=v6;")
A("        end")
A("    endtask")
A("")
A("    // ---------------- stimulus tasks ----------------")
A("    task do_reset;")
A("        begin")
A("            rst <= 1;")
A("            repeat (4) @(posedge clk);")
A("            rst <= 0;")
A("            @(posedge clk);")
A("        end")
A("    endtask")
A("")
A("    task load_kernels;                    // 27 bytes: kernel1, kernel2,
kernel3")
A("        integer k;")
A("        begin")
A("            for (k = 0; k < 27; k = k + 1) begin")
A("                @(posedge clk);")
A("                k_valid <= 1;  k_data <= kern[k];")
A("            end")
A("            @(posedge clk);  k_valid <= 0;")
A("        end")
A("    endtask")
A("")
A("    task pulse_start;")
A("        begin")
A("            @(posedge clk);  start <= 1;")
A("            @(posedge clk);  start <= 0;")
A("        end")
A("    endtask")
A("")
A("    task stream_image;                    // one pixel per clock,
row-major")
A("        integer p;")
A("        begin")
A("            for (p = 0; p < N*N; p = p + 1) begin")
A("                @(posedge clk);")
A("                pix_valid <= 1;  pix_data <= img[p];")
A("            end")
A("            @(posedge clk);  pix_valid <= 0;")
A("        end")
A("    endtask")
A("")
A("    // ---------------- output monitor ----------------")
A("    // Every cycle with out_valid=1 delivers one pooled position for all 3
channels.")
A("    always @(posedge clk) begin")
A("        if (out_valid) begin")
A("            if (n_out >= NOUT) begin")
A("                $display(\"ERROR: more than %0d outputs\", NOUT);")
A("                errors = errors + 1;")
A("            end else begin")
A("                got_val[0*NOUT+n_out] = out_data[7:0];")
A("                got_val[1*NOUT+n_out] = out_data[15:8];")
A("                got_val[2*NOUT+n_out] = out_data[23:16];")
A("                for (ch = 0; ch < 3; ch = ch + 1)")
A("                    if (got_val[ch*NOUT+n_out] !==
exp_val[ch*NOUT+n_out]) begin")
A("                        $display(\"MISMATCH ch%0d (r=%0d,c=%0d):
got %0d, expected %0d\",")
A("                                 ch+1, n_out/POOLN,
n_out%POOLN,")
A("                                 got_val[ch*NOUT+n_out],
exp_val[ch*NOUT+n_out]);")
A("                        errors = errors + 1;")
A("                    end")
A("                n_out = n_out + 1;")
A("            end")
A("        end")
A("    end")
A("")
A("    // ---------------- result printing ----------------")
A("    task print_grids;")
A("        begin")
A("            for (ch = 0; ch < 3; ch = ch + 1) begin")
A("                $display(\"\\nChannel %0d   (DUT output | expected)\",
ch+1);")
A("                for (r = 0; r < POOLN; r = r + 1) begin")
A("                    $write(\"  \");")
```

```
A("                    FOR (c = 0; c < POOLN; c = c + 1)
$WRITE(\"%3D \", GOT_VAL[CH*NOUT+R*POOLN+C]);")
A("                    $WRITE(\"   |   \");")
A("                    FOR (c = 0; c < POOLN; c = c + 1)
$WRITE(\"%3D \", EXP_VAL[CH*NOUT+R*POOLN+C]);")
A("                    $WRITE(\"\\N\");")
A("                END")
A("            END")
A("        END")
A("    ENDTASK")
A("")
A("    // ---------------- TIMEOUT GUARD ----------------")
A("    INITIAL BEGIN")
A("        #1_000_000;")
A("        $DISPLAY(\"TIMEOUT: DONE NEVER ASSERTED\");")
A("        $FINISH;")
A("    END")
A("")
A("    //
================================================================
")
A("    //  TEST DATA  (GENERATED FROM THE GOLDEN MODEL)")
A("    //
================================================================
")
A("    TASK FILL_TEST_DATA;")
A("        BEGIN")
A("            // ---- INPUT IMAGE 16x16 ----")
FOR R IN RANGE(16):
    A(F"            SET_ROW({R:2D}, {ROW_ARGS(IMG[R])});")
A("")
FOR K IN RANGE(3):
    A(F"            // ---- KERNEL {K+1} ----")
    KV = KERS[K].FLATTEN()
    A(F"            SET_KERNEL({K}, {ROW_ARGS(KV[0:3],4)},")
    A(F"                       {ROW_ARGS(KV[3:6],4)},")
    A(F"                       {ROW_ARGS(KV[6:9],4)});")
A("")
FOR CH IN RANGE(3):
    A(F"            // ---- EXPECTED OUTPUT, CHANNEL {CH+1} (AFTER CONV,
MAXPOOL, RELU, >> SHIFT, SATURATE) ----")
    FOR R IN RANGE(7):
        A(F"            SET_EXP({CH}, {R}, {ROW_ARGS(RELU_O[CH][R])});")
A("        END")
A("    ENDTASK")
A("")
A("    //
================================================================
")
A("    //  MAIN TEST SEQUENCE")
A("    //
================================================================
")
A("    INITIAL BEGIN")
A("        $DUMPFILE(\"TB_CUSTOM.VCD\");")
A("        $DUMPVARS(0, TB_CUSTOM);")
A("")
A("        FILL_TEST_DATA;")
A("        DO_RESET;                       // 1. RESET")
A("        LOAD_KERNELS;                   // 2. LOAD THE 3 KERNELS")
A("        PULSE_START;                    // 3. START")
A("        STREAM_IMAGE;                   // 4. SEND THE 256 PIXELS")
A("        WAIT (DONE);                    // 5. WAIT FOR THE DUT TO
FINISH")
A("        REPEAT (3) @(POSEDGE CLK);")
A("")
A("        PRINT_GRIDS;                    // 6. SHOW RESULTS")
A("")
A("        IF (N_OUT != NOUT) BEGIN")
A("            $DISPLAY(\"ERROR: GOT %0D OUTPUTS, EXPECTED %0D\", N_OUT,
NOUT);")
A("            ERRORS = ERRORS + 1;")
A("        END")
A("        IF (ERRORS == 0) $DISPLAY(\"\\NPASS: ALL %0D X 3 OUTPUTS MATCH
THE GOLDEN MODEL\", NOUT);")
A("        ELSE             $DISPLAY(\"\\NFAIL: %0D ERRORS\", ERRORS);")
A("        $FINISH;")
A("    END")
A("ENDMODULE")
OPEN("TB_CUSTOM.V", "W").WRITE("\N".JOIN(L) + "\N")
PRINT("\NWROTE TB_CUSTOM.V")
```

Tabel 2-2 Hasil Kode golden_custom.py

```
=== CHANNEL 1 ===
CONV (14x14), MIN/MAX = -25210 56877
[[  589  56877 -10722  27870  33899   2372  18981  -3476  30098   1903  18896  -9236
24068  -1770]
 [-13520  -4954  43023  17358   7080  20938  23633  -5136   -680   8999  30327  16545
14308   6300]
 [ 35043   9052  11795  16203  20010 -21709  21224  19368  14721   7589    238  36344
11000   7112]
 [ 14816  26358  16638  29035  24618  24404 -16870   1724  19261   2452  13758 -24165
23057  15197]
 [ 23458   4950   1961  20101 -14987  42342  -6727  11867  20790  24020  35226  17615
-7256  17588]
 [ -3522  12309  22614  28324  -2506  38048  -3231  11605   1718  -3008  35797 -10961
44101   4817]
 [  1795  16446  13086 -17566  27985  13297  -6354   7844  29911  29729   1941  -8021
11672  34906]
 [ 12953 -10103  48372  -1196  30446 -12987  12688  34767 -22282  20935   4150  42960
15798  17205]
 [ 14600  19088  -4910  -8664  28369  -5360 -13840  11094  24762  13122   8530   6159
13161  17357]
 [ 10763   7623  15035  36625   8224  41372  23977  10459   5786  -5966  28791   8788
17839 -25210]
 [ 13374   -610   9508  10750   3437  -1738  14768  12085  16395   4101  39453  -6788
20491  17382]
 [ 19829  11826  11707  15396  24579   2989  12547  21210  24383  10913  12955  17442
-5711   6963]
 [-11215  17603   3503  16575  16192   3257   5045  -7639  32192  34728 -24190   7975
6716   8481]
 [ 16295  18108  31560  -5773   -206  27355  18603  22212 -23104  29188  25175   7698
3113  28590]]
MAXPOOL (7X7):
[[56877 43023 33899 23633 30098 30327 24068]
 [35043 29035 24618 21224 19261 36344 23057]
 [23458 28324 42342 11867 24020 35797 44101]
 [16446 48372 30446 34767 29911 42960 34906]
 [19088 36625 41372 23977 24762 28791 17839]
 [19829 15396 24579 21210 24383 39453 20491]
 [18108 31560 27355 22212 34728 25175 28590]]
RELU+SAT (7X7):
[[222 168 132  92 117 118  94]
 [136 113  96  82  75 141  90]
 [ 91 110 165  46  93 139 172]
 [ 64 188 118 135 116 167 136]
 [ 74 143 161  93  96 112  69]
 [ 77  60  96  82  95 154  80]
 [ 70 123 106  86 135  98 111]]
=== CHANNEL 2 ===
CONV (14x14), MIN/MAX = -57031 7791
[[-36488 -38011 -13806 -28260 -17396 -46702 -35074 -17116 -21299 -12706 -27415   1719
-21366 -10182]
 [-37030 -28801 -31498 -34533 -42668 -33963 -44091 -18108 -14395 -30945 -17532 -33338
-5263 -30910]
 [ -1856  -9319 -32827 -21858 -57031 -30802 -54561 -17338 -11987 -33660 -23264 -40564
-19680 -12223]
 [-28809 -37225 -20105 -37507 -32708 -12536 -34823 -16812  -2321   7791 -20729 -44684
-28099   3409]
 [-31143 -28753 -19066 -45012 -27139 -42196 -37763 -32841   -809 -16575  -2302 -44245
-22049 -25818]
 [-20726 -34874  -2701 -37422 -19033 -44868 -36128 -13790 -37678  -8705 -48642 -16788
-45485  -3557]
 [  4248 -26089 -19643 -24670 -21903 -33629 -45042 -21870   7264 -24108 -10115 -41379
-20239 -32004]
 [-34079 -25888 -24352 -12598 -35665 -16598 -29309 -14707 -21039 -36626 -11978 -14158
-16959 -44796]
 [  -529 -11343 -24014  -5370  -5503  -6102 -26438 -29209  -9497 -34353 -13571 -41456
-40489 -33531]
 [-25503 -42628 -20343 -36541 -11694 -27708 -16472 -13094 -20998  -6646 -19317  -7337
-34685 -46252]
 [-37594  -9078 -25365 -26130 -33050 -40360 -23540 -28337 -16046  -5890 -41121 -13291
-28622 -12254]
 [-21570 -22230  -2260 -23789 -38449 -22173 -30905 -24798 -40933 -26997 -42771 -42946
-23980 -17666]
 [-20570 -33464    504 -23740 -26053 -19524 -32100 -12393 -31244 -26659 -33850 -28570
6568 -30849]
 [-24434 -24798 -23323 -24986 -31332 -35512   -557 -22752 -29637 -26257 -21947 -22942
5423  -6142]]
MAXPOOL (7X7):
[[-28801 -13806 -17396 -17116 -12706   1719  -5263]
 [ -1856 -20105 -12536 -16812   7791 -20729   3409]
 [-20726  -2701 -19033 -13790   -809  -2302  -3557]
 [  4248 -12598 -16598 -14707   7264 -10115 -16959]
 [  -529  -5370  -5503 -13094  -6646  -7337 -33531]
 [ -9078  -2260 -22173 -23540  -5890 -13291 -12254]
 [-20570    504 -19524   -557 -26257 -21947   6568]]
RELU+SAT (7X7):
[[ 0  0  0  0  0  6  0]
 [ 0  0  0  0 30  0 13]
 [ 0  0  0  0  0  0  0]
 [16  0  0  0 28  0  0]
 [ 0  0  0  0  0  0  0]
 [ 0  0  0  0  0  0  0]
 [ 0  1  0  0  0  0 25]]
=== CHANNEL 3 ===
CONV (14x14), MIN/MAX = -12177 58472
[[ -6162  57799  -2873  49328  24421  24519  50092  -2231  38106  10147  53006  17713
31484  14837]
 [ 25439  24629  38635  25143  28153  36194  41034  31516   6957   8790  33119  14349
38880  -6965]
 [ 31566  25817  43333  42213  41082  18485  22841  28074  13829  12578   7913  37636
33591  36264]
 [ 11168  23297  11278  22251  44335  43663  21930  48594  32367  26508  35798  13444
43179   6921]
 [ 23066  42627  26362  40404  11238  47381   -861  20070   1038   6255  39562  13794
```

```
36362  17249]
 [ 5327  15152  25515  17739  24342  43268  24616  47406  17637  31805  14417  17409
40739  33914]
 [ -647  43808  43292   8448  35414  15578  20909  21677  19096  22419  48985   7947
52866  16233]
 [ 12566   8813  26018  -6352  46863  -2288  23326  43895  -1904  43209  -9214  44047
15402  58472]
 [ 35771  23797  42253  37004  51424  21031  25811  24320  19980  11833  37230   8223
29584   5575]
 [-12177  17442   3452   8872   -536  22812  29641  26435  27292  26159  40515  33921
36386  18830]
 [ 43531  39313  26002  45547  29433  34137  27733  20016  35062   5947  42953   5042
27913  26815]
 [ 15346  17135  18721   8401  40707   9122  15330  26452  30229  29201  15123  31373
9439   8881]
 [ 15629  51236  20779  27341  39283  33965  34115   9921  30267  51625  18881  49187
16170  22576]
 [ 14479  42520  46188  14851  12729  26866  24739  19557   2695  30338  18263  34508
21073  27177]]
MAXPOOL (7X7):
[[57799 49328 36194 50092 38106 53006 38880]
 [31566 43333 44335 48594 32367 37636 43179]
 [42627 40404 47381 47406 31805 39562 40739]
 [43808 43292 46863 43895 43209 48985 58472]
 [35771 42253 51424 29641 27292 40515 36386]
 [43531 45547 40707 27733 35062 42953 27913]
 [51236 46188 39283 34115 51625 49187 27177]]
RELU+SAT (7X7):
[[225 192 141 195 148 207 151]
 [123 169 173 189 126 147 168]
 [166 157 185 185 124 154 159]
 [171 169 183 171 168 191 228]
 [139 165 200 115 106 158 142]
 [170 177 159 108 136 167 109]
 [200 180 153 133 201 192 106]]
```

2.2 TESTBENCH DAN HASILNYA

Tabel 2-3 Kode tb_custom.v

```
`timescale 1ns/1ps
//
================================================================
====
//  tb_custom.v  --  directed test: Conv(3 kernels) + MaxPool 2x2
+ ReLU
//  AUTO-GENERATED by golden_custom.py (edit the numbers there,
not here)
//
//  Test flow:  reset -> load 3 kernels -> start -> stream 16x16
image
//              -> collect 7x7x3 outputs -> compare with golden
model
//
================================================================
====
module tb_custom;

    // ---------------- constants ----------------
    localparam N     = 16;          // image size
    localparam POOLN = 7;           // output size per channel
    localparam NOUT  = POOLN*POOLN; // 49 outputs per channel
    localparam SHIFT = 8;          // output quantization: ReLU,
>> SHIFT, saturate

    // ---------------- DUT signals ----------------
    reg         clk = 0, rst = 1, start = 0;
    reg         k_valid = 0, pix_valid = 0;
    reg  [7:0]  k_data = 0, pix_data = 0;
    wire        out_valid, done;
    wire [23:0] out_data;           // {ch3, ch2, ch1}

    conv_pool_relu_top #(.N(N), .SHIFT(SHIFT)) dut (
        .clk(clk), .rst(rst), .start(start),
        .k_valid(k_valid), .k_data(k_data),
        .pix_valid(pix_valid), .pix_data(pix_data),
        .out_valid(out_valid), .out_data(out_data), .done(done));

    always #5 clk = ~clk;           // 100 MHz

    // ---------------- test data storage ----------------
    reg [7:0] img      [0:N*N-1];       // image, row-major
    reg [7:0] kern     [0:26];          // 3 kernels x 9 weights
(two's complement)
    reg [7:0] exp_val  [0:3*NOUT-1];    // expected, index =
ch*49 + r*7 + c
    reg [7:0] got_val  [0:3*NOUT-1];    // captured from the DUT
    integer i, ch, r, c;
    integer n_out  = 0;
    integer errors = 0;

    // ---------------- helper tasks to fill the data
----------------
    task set_row(input integer row,
                 input integer p0, p1, p2, p3, p4, p5, p6, p7,
                 input integer p8, p9, p10, p11, p12, p13, p14,
p15);
        begin
            img[row*N+0]=p0;   img[row*N+1]=p1;
img[row*N+2]=p2;   img[row*N+3]=p3;
            img[row*N+4]=p4;   img[row*N+5]=p5;
img[row*N+6]=p6;   img[row*N+7]=p7;
            img[row*N+8]=p8;   img[row*N+9]=p9;
img[row*N+10]=p10; img[row*N+11]=p11;
            img[row*N+12]=p12; img[row*N+13]=p13;
img[row*N+14]=p14; img[row*N+15]=p15;
        end
    endtask

    task set_kernel(input integer k,
                    input integer a, b, c3,
                    input integer d, e, f,
                    input integer g, h, i3);
        begin
            kern[k*9+0]=a; kern[k*9+1]=b; kern[k*9+2]=c3;
            kern[k*9+3]=d; kern[k*9+4]=e; kern[k*9+5]=f;
            kern[k*9+6]=g; kern[k*9+7]=h; kern[k*9+8]=i3;
        end
    endtask

    task set_exp(input integer chn, input integer row,
                 input integer v0, v1, v2, v3, v4, v5, v6);
        begin
            exp_val[chn*NOUT+row*POOLN+0]=v0;
exp_val[chn*NOUT+row*POOLN+1]=v1;
            exp_val[chn*NOUT+row*POOLN+2]=v2;
exp_val[chn*NOUT+row*POOLN+3]=v3;
            exp_val[chn*NOUT+row*POOLN+4]=v4;
exp_val[chn*NOUT+row*POOLN+5]=v5;
            exp_val[chn*NOUT+row*POOLN+6]=v6;
        end
    endtask

    // ---------------- stimulus tasks ----------------
    task do_reset;
        begin
            rst <= 1;
            repeat (4) @(posedge clk);
            rst <= 0;
            @(posedge clk);
        end
    endtask

    task load_kernels;                  // 27 bytes: kernel1,
kernel2, kernel3
        integer k;
        begin
            for (k = 0; k < 27; k = k + 1) begin
                @(posedge clk);
                k_valid <= 1;  k_data <= kern[k];
            end
            @(posedge clk);  k_valid <= 0;
        end
    endtask

    task pulse_start;
        begin
            @(posedge clk);  start <= 1;
            @(posedge clk);  start <= 0;
        end
    endtask

    task stream_image;                  // one pixel per clock,
row-major
        integer p;
        begin
            for (p = 0; p < N*N; p = p + 1) begin
                @(posedge clk);
                pix_valid <= 1;  pix_data <= img[p];
            end
            @(posedge clk);  pix_valid <= 0;
        end
```

```
    endtask

    // ---------------- output monitor ----------------
    // Every cycle with out_valid=1 delivers one pooled position
for all 3 channels.
    always @(posedge clk) begin
        if (out_valid) begin
            if (n_out >= NOUT) begin
                $display("ERROR: more than %0d outputs", NOUT);
                errors = errors + 1;
            end else begin
                got_val[0*NOUT+n_out] = out_data[7:0];
                got_val[1*NOUT+n_out] = out_data[15:8];
                got_val[2*NOUT+n_out] = out_data[23:16];
                for (ch = 0; ch < 3; ch = ch + 1)
                    if (got_val[ch*NOUT+n_out] !==
exp_val[ch*NOUT+n_out]) begin
                        $display("MISMATCH ch%0d (r=%0d,c=%0d):
got %0d, expected %0d",
                                 ch+1, n_out/POOLN, n_out%POOLN,
                                 got_val[ch*NOUT+n_out],
exp_val[ch*NOUT+n_out]);
                        errors = errors + 1;
                    end
            end
            n_out = n_out + 1;
        end
    end

    // ---------------- result printing ----------------
    task print_grids;
        begin
            for (ch = 0; ch < 3; ch = ch + 1) begin
                $display("\nChannel %0d   (DUT output |
expected)", ch+1);
                for (r = 0; r < POOLN; r = r + 1) begin
                    $write("  ");
                    for (c = 0; c < POOLN; c = c + 1) $write("%3d
", got_val[ch*NOUT+r*POOLN+c]);
                    $write("   |   ");
                    for (c = 0; c < POOLN; c = c + 1) $write("%3d
", exp_val[ch*NOUT+r*POOLN+c]);
                    $write("\n");
                end
            end
        end
    endtask

    // ---------------- timeout guard ----------------
    initial begin
        #1_000_000;
        $display("TIMEOUT: done never asserted");
        $finish;
    end

    //
================================================================
    //  TEST DATA  (generated from the golden model)
    //
================================================================
    task fill_test_data;
        begin
            // ---- input image 16x16 ----
            set_row( 0, 170, 249,  18, 218,   4, 172, 206,  45,
191,  76, 195,   6, 103,  89, 209, 163);
            set_row( 1, 234, 241, 143,  93, 229, 230, 158, 119,
115, 156, 111,  20,  95,  31, 217,  94);
            set_row( 2,  26,  54, 185, 164, 199, 220, 219,  90,
40, 107, 192, 212,  92, 103,  95, 202);
            set_row( 3,  92,  91,  85, 180, 168, 129, 183, 231,
23,  32, 161, 184, 206,   4, 103,  45);
            set_row( 4,  79, 179, 195, 219, 162, 231,  28, 167,
13,  29,  53,  25, 246, 145,  93,  76);
            set_row( 5,  49, 148, 105,  42, 178, 213, 138, 247,
113, 150,  91, 186, 212,  76, 123, 235);
            set_row( 6,  24, 221,  81,  72, 212, 109, 200, 162,
0,  41, 171, 154, 251,  12, 179, 192);
            set_row( 7,  97,  80,  54,  30, 214,  28, 225, 131,
53, 201,   0, 165,  36, 223, 109, 211);
            set_row( 8,  20,  73, 245, 167,  66,  17, 201, 114,
27,  94, 203, 117, 182, 105, 188,  86);
            set_row( 9,  13, 219,  64,  40,  42, 103,  36,  71,
151, 151, 104,  37, 149, 246, 240,  57);
            set_row(10, 218, 119, 146, 218, 193, 238, 155, 134,
118,  34, 115,  23, 184,  64,  80, 110);
            set_row(11,  26,  51,  82,  69, 132,  52, 198, 169,
115, 127, 142, 212, 139, 131,  73,   3);
            set_row(12,  65, 222,  63, 119, 129, 151, 156, 114,
187, 150, 158, 253,  45, 125, 124,  93);
            set_row(13,  88, 199,  75,  42, 189, 147,   5,  50,
162, 214,  64, 122,  53,  33,  73, 152);
            set_row(14, 233, 217, 153,  83, 216, 174, 112, 111,
6, 184, 204, 169,  65,  62, 204,  76);
            set_row(15, 156, 215, 248, 246, 127,  75, 156,  53,
91,  50, 108, 239, 185,  92, 231, 223);

            // ---- kernel 1 ----
            set_kernel(0,  -28, -111,  125,
                           116,    4,   -3,
                           -90,   76,    7);
            // ---- kernel 2 ----
            set_kernel(1, -100,  -38,    7,
                           -53,  -45,  -47,
                            27,  -64,  122);
            // ---- kernel 3 ----
            set_kernel(2,  105,  -76,  115,
                             2,  -40,  -40,
                            59,   60,   16);

            // ---- expected output, channel 1 (after conv,
maxpool, ReLU, >> SHIFT, saturate) ----
            set_exp(0, 0, 222, 168, 132,  92, 117, 118,  94);
            set_exp(0, 1, 136, 113,  96,  82,  75, 141,  90);
            set_exp(0, 2,  91, 110, 165,  46,  93, 139, 172);
            set_exp(0, 3,  64, 188, 118, 135, 116, 167, 136);
            set_exp(0, 4,  74, 143, 161,  93,  96, 112,  69);
            set_exp(0, 5,  77,  60,  96,  82,  95, 154,  80);
            set_exp(0, 6,  70, 123, 106,  86, 135,  98, 111);
            // ---- expected output, channel 2 (after conv,
maxpool, ReLU, >> SHIFT, saturate) ----
            set_exp(1, 0,   0,   0,   0,   0,   0,   6,   0);
            set_exp(1, 1,   0,   0,   0,   0,  30,   0,  13);
            set_exp(1, 2,   0,   0,   0,   0,   0,   0,   0);
            set_exp(1, 3,  16,   0,   0,   0,  28,   0,   0);
            set_exp(1, 4,   0,   0,   0,   0,   0,   0,   0);
            set_exp(1, 5,   0,   0,   0,   0,   0,   0,   0);
            set_exp(1, 6,   0,   1,   0,   0,   0,   0,  25);
            // ---- expected output, channel 3 (after conv,
maxpool, ReLU, >> SHIFT, saturate) ----
            set_exp(2, 0, 225, 192, 141, 195, 148, 207, 151);
            set_exp(2, 1, 123, 169, 173, 189, 126, 147, 168);
            set_exp(2, 2, 166, 157, 185, 185, 124, 154, 159);
            set_exp(2, 3, 171, 169, 183, 171, 168, 191, 228);
            set_exp(2, 4, 139, 165, 200, 115, 106, 158, 142);
            set_exp(2, 5, 170, 177, 159, 108, 136, 167, 109);
            set_exp(2, 6, 200, 180, 153, 133, 201, 192, 106);
        end
    endtask

    //
================================================================
    //  MAIN TEST SEQUENCE
    //
================================================================
    initial begin
        $dumpfile("tb_custom.vcd");
        $dumpvars(0, tb_custom);

        fill_test_data;
        do_reset;                   // 1. reset
        load_kernels;               // 2. load the 3 kernels
        pulse_start;                // 3. start
        stream_image;               // 4. send the 256 pixels
        wait (done);                // 5. wait for the DUT to
finish
        repeat (3) @(posedge clk);

        print_grids;                // 6. show results

        if (n_out != NOUT) begin
            $display("ERROR: got %0d outputs, expected %0d",
n_out, NOUT);
            errors = errors + 1;
        end
        if (errors == 0) $display("\nPASS: all %0d x 3 outputs
match the golden model", NOUT);
        else             $display("\nFAIL: %0d errors", errors);
        $finish;
    end
endmodule
```

Tabel 2-4 Hasil Kode tb_custom.v

```
CHANNEL 1  (DUT OUTPUT | EXPECTED)
 222 168 132  92 117 118  94   |  222 168 132  92 117 118  94
 136 113  96  82  75 141  90   |  136 113  96  82  75 141  90
  91 110 165  46  93 139 172   |   91 110 165  46  93 139 172
  64 188 118 135 116 167 136   |   64 188 118 135 116 167 136
  74 143 161  93  96 112  69   |   74 143 161  93  96 112  69
  77  60  96  82  95 154  80   |   77  60  96  82  95 154  80
  70 123 106  86 135  98 111   |   70 123 106  86 135  98 111

CHANNEL 2  (DUT OUTPUT | EXPECTED)
   0   0   0   0   0   6   0   |    0   0   0   0   0   6   0
   0   0   0   0  30   0  13   |    0   0   0   0  30   0  13
   0   0   0   0   0   0   0   |    0   0   0   0   0   0   0
  16   0   0   0  28   0   0   |   16   0   0   0  28   0   0
   0   0   0   0   0   0   0   |    0   0   0   0   0   0   0
   0   0   0   0   0   0   0   |    0   0   0   0   0   0   0
   0   1   0   0   0   0  25   |    0   1   0   0   0   0  25

CHANNEL 3  (DUT OUTPUT | EXPECTED)
 225 192 141 195 148 207 151   |  225 192 141 195 148 207 151
 123 169 173 189 126 147 168   |  123 169 173 189 126 147 168
 166 157 185 185 124 154 159   |  166 157 185 185 124 154 159
 171 169 183 171 168 191 228   |  171 169 183 171 168 191 228
 139 165 200 115 106 158 142   |  139 165 200 115 106 158 142
 170 177 159 108 136 167 109   |  170 177 159 108 136 167 109
 200 180 153 133 201 192 106   |  200 180 153 133 201 192 106
```

2.3 HASIL WAVEFORM



Gambar 2-1 Hasil Waveform Kernel Loading



Gambar 4-2 Hasil Waveform untuk RUN hingga DONE

2.4 VERIFIKASI GELOMBANG

Dapat dilihat bahwa waveform mengikuti FSM yang telah dibuat, yaitu:

1. Saat IDLE, sistem melakukan *loading kernel* hingga keseluruhan kernel di load.
2. Saat RUN, sistem melakukan *loading* untuk input dan sekaligus menghasilkan output saat nilai yang dibutuhkan sudah cukup.
3. Saat DONE, seluruh output telah dikeluarkan dan kembali lagi ke state IDLE.

2.5 PERBANDINGAN HASIL TESTBENCH DAN GOLDEN MODEL

Dapat dilihat bahwa hasil dari testbench dan golden model sudah sama dengan keduanya mendapatkan hasil dari ketiga channel yang sama.

3. Kesimpulan dan Pembahasan

Dari sistem yang dirancang dapat disimpulkan bahwa:

1. Hasil dari sistem telah sesuai dengan *golden model* yang telah dibuat dengan hasil dari keduanya mendapatkan hasil yang sama.
2. Hasil *waveform* memperlihatkan kerja FSM yang sesuai dan memperlihatkan optimisasi dari area dan *throughput* yang telah dibuat.

Daftar Pustaka

[1] ChipVerify, "Verilog Tutorial," ChipVerify. [Online]. Available: https://chipverify.com/tutorials/verilog. [Accessed: Oct. 11, 2026].

[2] The DataBus, "The Convolution Engine," thedatabus.in. [Online]. Available: https://thedatabus.in/convolver/. [Accessed: Oct. 11, 2026].